

Structure and Interpretation of Computer Programs (SICP)

Anotações pessoais de estudo

Miquéias Alves Medeiros

17 de setembro de 2026

Anotações elaboradas a partir da obra de Harold Abelson, Gerald Jay Sussman e Julie Sussman.

Sumário

1	Anotações do cap. 1 Building Abstractions with Procedures, pp. 10–11	2
1.1	Naming and the Environment	2
1.2	More Examples of <code>define</code>	2
1.3	The Role of Naming Computational Objects	2
1.4	The Environment	3
	Referências	4

1 Anotações do cap. 1 Building Abstractions with Procedures, pp. 10–11

1.1 Naming and the Environment

In programming languages, we can assign names to computational objects. These names refer to variables that hold the value of this object. In the Scheme dialect of Lisp, we can declare variables using `define`, typing:

```
1 (define size 2)
```

This tells the interpreter to associate the value 2 with the name `size`. In mathematical notation, this can be expressed as:

$$\text{size} = 2$$

Once it is declared, we can refer to the value 2 by name and perform operations with it:

```
1 (* 5 size) ; 10
```

The interpreter evaluates the expression to 10.

1.2 More Examples of `define`

We can define variables to perform calculations involving computational objects.

```
1 (define pi 3.14159)
```

```
2 (define radius 10)
```

Here, we are calculating the area of a circle. We can perform a squaring operation by multiplying the radius variable by itself to obtain r^2 .

```
1 (* pi (* radius radius)) ; 314.159
```

We can get the circumference as well using the same idea. A variable can be defined and assigned the result of a calculation.

```
1 (define circumference (* 2 pi radius)) ; 62.8318
```

1.3 The Role of Naming Computational Objects

Naming computational objects allows us to perform complex compound operations without trying to remember and repeat the details of the computational objects used in those operations.

These objects may have very complex structures, and it would be extremely tedious to remember all of that every time we used those structures.

In addition, complex programs are created by building, step by step, computational objects of increasing complexity. The interpreter makes this gradual program construction particularly convenient, as associations between names and objects can be created incrementally across successive interactions.

This feature encourages the incremental development and testing of programs and is largely responsible for the fact that a Lisp program typically consists of a large number of relatively simple procedures.

1.4 The Environment

Thus, it is clear that some form of memory is required for this association of values with symbols to be possible, and for those values to be retrievable.

This memory is called an *environment*, more precisely, the *global environment*.

Referências

Referências

- [1] ABELSON, Harold; SUSSMAN, Gerald Jay; SUSSMAN, Julie. *Structure and Interpretation of Computer Programs*. 2. ed. Cambridge, MA: MIT Press, 1996.